

SatQuery AI: Architecture Blueprint

An Interactive Vision-Language Assistant for Multimodal Remote Sensing Image Analysis
Target Problem Statement: SIH ISRO PS 26167

1. Executive Summary & Core Challenges

Traditional Indian satellite platforms (such as the ISRO Bhuvan and Bhoonidhi data portals) host incredibly massive archives of high-value Earth Observation (EO) data. However, converting these raw rasters into real-world insights demands deep Geographic Information Systems (GIS) domain expertise, manual multispectral band operations, and fragmented single-purpose neural network pipelines.

SatQuery AI overcomes this operational bottleneck by serving as a unified conversational intelligence, translation layer, and spatial grounding engine. Non-technical users—ranging from agricultural analysts to frontline disaster relief coordinators—can issue loose, plain-text queries and receive context-aware summaries completely mapped to localized geographic coordinate systems.

2. The Pipeline-Fusion Hybrid Architecture

To deliver production-grade robustness far outperforming standard monolithic vision-language wrappers, this framework adopts a **Pipeline-Fusion Hybrid Architecture**. By leveraging decoupled multi-sensor routing alongside classical mathematical verification layers, the framework ensures absolute analytical precision.

Architectural Dimension	Traditional Frameworks	SatQuery AI Hybrid Pipeline
User Barrier	High; mandates GIS expertise, custom scripting, manual band combinations.	Zero; accessible via plain, natural conversational prompts.
Task Versatility	Hardcoded single-task models (e.g., rigid classifiers, separate object counters).	Dynamic multi-tasking orchestrator routing code dynamically.
Spectral Ingestion	Strictly limited to standard 3-band RGB visual presentations.	Native multi-spectral intelligence (SAR, SWIR, NIR band parsing).
System Outputs	Static flat maps or disconnected tabular CSV files.	Simultaneous Interactive Overlays + Conversational Briefs + Structured GeoJSON.

3. Deep-Dive Process Pipeline Blocks

3.1 Language & Workflow Orchestration Block

- **Query Understanding & Blueprint Assembly:** A localized Large Language Model (e.g., LLaMA-3 or DeepSeek-Coder fine-tuned on structural GIS prompts) acts as the operational nerve center. It fragments natural text queries into parameter maps (spatial bounds, temporal constraints, and target analytic goals) before triggering any resource-intensive geospatial loads.
- **Multi-Sensor Structural Branching:** The workflow builder assigns tasks into separate parallel tracks depending on criteria like cloud cover or night status:
 - *Optical Track:* Ingests multi-spectral data (Landsat/Sentinel/Resourcesat) for land cover, vegetation, and high-res urban visuals.
 - *SAR Track:* Ingests Synthetic Aperture Radar assets (RISAT/Sentinel-1). Essential for penetrating heavy cloud blocks or storms during active flooding events.
 - *Temporal History:* Pulls historically accurate background tiles to map change-detection patterns over a continuous timeline.

3.2 Execution & Evidence Fusion Block

- **Specialist Deep Learning Agents:** High-throughput deep learning models run targeted extractions. It pairs *YOLO-World / Grounding DINO* for open-vocabulary grounding (unlisted objects) alongside *SAM-Geo* (*Segment Anything Model for Geospatial Data*) for boundaries.
- **Local Physics GIS Engine:** Concurrently runs programmatic mathematical equations using libraries like *rasterio* and *geopandas* to map objective truth matrices like NDVI (Normalized Difference Vegetation Index) or NDWI (Water Index).
- **Evidence Fusion & Verifier:** The core differentiator. Predictions made by neural networks are explicitly cross-validated against physics indices. For instance, if an agent flags an area as 'flooded,' the verifier ensures its mathematical NDWI values strongly match a water body signature before certifying the output.

4. Structural Backend Payload Blueprint

To guarantee seamless structural compatibility with production GIS applications or ISRO's Bhuvan platform, the backend API maps execution results into unified JSON structures carrying automated auditing metadata.

```
{
  "query_id": "sq_90812",
  "task_type": "Object_Detection_And_Index_Analysis",
  "summary": "12 storage tanks detected. Average vegetation index (NDVI) is 0.15 indicating dry land.",
  "metrics": { "processing_time_ms": 420, "confidence_score": 0.94 },
  "geospatial_data": {
    "crs": "EPSG:4326",
    "detected_objects": [
      {
        "label": "Storage Tank",
        "confidence": 0.96,
```

```
      "geometry": {
        "type": "Polygon",
        "coordinates": [[[74.821, 21.345], [74.823, 21.345], [74.823, 21.347], [74.821, 21.347], [74.821, 21.345]]]
      }
    ]
  }
}
```

5. Implementation Roadmap

1. **Inundation & Vector Ingestion:** Implement asynchronous streaming files via Python, processing bands via `rioxarray` and `GDAL` environments.
2. **Orchestrator Fine-Tuning:** Configure small open-weight VLMs (e.g., Qwen2-VL) on paired image-coordinate instruction sets like EuroSAT, SpaceNet, and LEVIR-CD.
3. **Map Engine Frontend:** Bind UI containers using Next.js interacting natively with Mapbox GL JS/Leaflet to stream live GeoJSON polygons smoothly.